

Algoritma Analizi - Odev 3

Algoritma Analizi ve Tasarımı — Ödev 3 Cevapları

Ad-Soyad: Samed Bilgin Numara: —

Soru 1 (20 puan) — İkili Arama Ağacı (BST) ve Gezintiler

Verilen ekleme sırası: **n, z, d, f, a, k, o, m, e, l, t, g**

Adım Adım Ekleme

Adım	Eklenen	Karar
1	n	Kök
2	z	$z > n \rightarrow n$ 'in sağ
3	d	$d < n \rightarrow n$ 'in sol
4	f	$f < n, f > d \rightarrow d$ 'nin sağ
5	a	$a < n, a < d \rightarrow d$ 'nin sol
6	k	$k < n, k > d, k > f \rightarrow f$ 'nin sağ
7	o	$o > n, o < z \rightarrow z$ 'nin sol
8	m	$m < n, m > d, m > f, m > k \rightarrow k$ 'nin sağ
9	e	$e < n, e > d, e < f \rightarrow f$ 'nin sol
10	l	$l < n, l > d, l > f, l > k, l < m \rightarrow m$ 'nin sol
11	t	$t > n, t < z, t > o \rightarrow o$ 'nun sağ
12	g	$g < n, g > d, g > f, g < k \rightarrow k$ 'nin sol

Oluşan İkili Arama Ağacı

```
      n
     / \
    d   z
   / \ /
  a  f o
   / \ \
  e  k t
     / \
    g  m
     /
    l
```

Gezintiler (Traversals)

- Preorder** (Kök → Sol → Sağ): **n, d, a, f, e, k, g, m, l, z, o, t**

- **Inorder** (Sol → Kök → Sağ): **a, d, e, f, g, k, l, m, n, o, t, z**
(Beklendiği gibi alfabetik sıralı, çünkü BST in-order = sıralı çıktı verir.)
- **Postorder** (Sol → Sağ → Kök): **a, e, g, l, m, k, f, d, t, o, z, n**

Soru 2 (20 puan) — DFS Tabanlı Topological Sort

Not: Aşağıda her grafin okunan kenar listesi ile birlikte çözüm verilmiştir. Grafiğinizde farklı bir kenar varsa, aynı algoritma akışını kendi kenar listenize uygulayın.

Graf (a)

Kenarlar: a→b, a→f, d→a, d→b, d→c, d→f, d→g, b→e, e→g, g→f

In-degree analizi: - a: 1 (d'den) - b: 2 (a, d) - c: 1 (d) - d: **0** (kaynak) - e: 1 (b) - f: 3 (a, d, g) - g: 2 (d, e)

Hiçbir düğüm bir back-edge oluşturmuyor (d'ye gelen kenar yok), dolayısıyla **döngü yoktur → DAG**, topolojik sıralama mümkündür.

DFS (kaynak = d, alfabetik komşu sırası):

DFS(d):

DFS(a):

DFS(b):

DFS(e):

DFS(g):

DFS(f) – finish f → yığın: [f]

finish g → yığın: [f, g]

finish e → yığın: [f, g, e]

finish b → yığın: [f, g, e, b]

DFS(f) zaten ziyaret

finish a → yığın: [f, g, e, b, a]

DFS(b) zaten ziyaret

DFS(c) – finish c → yığın: [f, g, e, b, a, c]

DFS(f), DFS(g) zaten ziyaret

finish d → yığın: [f, g, e, b, a, c, d]

Bitiş zamanlarının **tersi** topolojik sıralamayı verir:

Topological order (a): d → c → a → b → e → g → f

Graf (b)

Kenarlar: a→b, b→c, c→d, a→e, b→f, c→f, e→f, f→g, d→g

In-degree analizi: - a: **0** (kaynak) - b: 1 (a) - c: 1 (b) - d: 1 (c) - e: 1 (a) - f: 3 (b, c, e) - g: 2 (d, f)

Geri kenar yok → **DAG**, topolojik sıralama mümkündür.

DFS (kaynak = a, alfabetik komşu sırası):

DFS(a):

DFS(b):

DFS(c):

DFS(d):

```
DFS(g) – finish g      → yığın: [g]
finish d                → yığın: [g, d]
DFS(f):
  DFS(g) zaten
  finish f              → yığın: [g, d, f]
  finish c              → yığın: [g, d, f, c]
DFS(f) zaten
finish b                → yığın: [g, d, f, c, b]
DFS(e):
  DFS(f) zaten
  finish e              → yığın: [g, d, f, c, b, e]
  finish a              → yığın: [g, d, f, c, b, e, a]
```

Bitiş zamanlarının tersi:

Topological order (b): $a \rightarrow e \rightarrow b \rightarrow c \rightarrow f \rightarrow d \rightarrow g$

Doğrulama: bütün kenarlar ($u \rightarrow v$) için u, sıralamada v’den önce gelir ✓.

Döngü olsaydı ne olurdu?

DFS sırasında, **henüz finish olmamış (gri renkli) bir düğüme tekrar gidilmesi back-edge’dir**. Back-edge keşfedilirse graf döngü içerir → **topolojik sıralama tanımsızdır**, çünkü topolojik sıralama tüm kenarları “geriye dönmeyecek” şekilde dizmeyi gerektirir; döngüde her düğüm hem önce hem sonra olmak zorunda kalır ki bu çelişir.

Soru 3 (60 puan) — BST, AVL, 2-3 Ağacı Karşılaştırması

Kod: [q3_trees.py](#) (Python 3)

Çalıştırma

```
python3 q3_trees.py
```

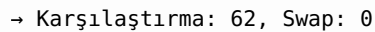
Örnek Çıktı (Tek Çalıştırma)

```
=====
VERİ SETİ (0-1000 arası 20 rastgele sayı, bir defa üretildi)
=====
```

```
[294, 491, 25, 168, 7, 242, 895, 519, 255, 293, 229, 467, 572, 165,
714, 442, 172, 940, 99, 589]
```

```
=====
BINARY SEARCH TREE
=====
```

```
└─ root: 294
  └─ L: 25
    └─ L: 7
      └─ R: 168
        └─ L: 165
```



AVL TREE

```

└─ root: 255 (h=5)
  └─ L: 168 (h=4)
    └─ L: 25 (h=3)
      └─ L: 7 (h=1)
        └─ R: 165 (h=2)
          └─ L: 99 (h=1)
            └─ R: .
          └─ R: 229 (h=2)
            └─ L: 172 (h=1)
              └─ R: 242 (h=1)
            └─ R: 491 (h=4)
              └─ L: 294 (h=3)
                └─ L: 293 (h=1)
                  └─ R: 467 (h=2)
                    └─ L: 442 (h=1)
                      └─ R: .
                    └─ R: 714 (h=3)
                      └─ L: 572 (h=2)
                        └─ L: 519 (h=1)
                          └─ R: 589 (h=1)
                        └─ R: 895 (h=2)
                          └─ L: .
                          └─ R: 940 (h=1)

```

=====

```
└─ root: [294]
    └─ C0: [168]
        └─ C0: [25]
            └─ C0: [7]
                └─ C1: [99, 165]
                    └─ C1: [242]
                        └─ C0: [172, 229]
                            └─ C1: [255, 293]
                                └─ C1: [519]
                                    └─ C0: [467]
                                        └─ C0: [442]
                                            └─ C1: [491]
                                                └─ C1: [714]
                                                    └─ C0: [572, 589]
                                                        └─ C1: [895, 940]
```

→ Karşılaştırma: 67, Split (swap): 11

Karşılaştırma Tablosu

Veri (özet)	Binary Search Tree	AVL Tree	2-3 Tree
294, 491, 25, 168, 7, 242, 895, 519, 255, 293, 229, 467, 572, 165, 714, 442, 172, 940, 99, 589	62 karşılaştırma, 0 swap	63 karşılaştırma, 14 rotation	67 karşılaştırma, 11 split

Yorum

- **BST:** hiçbir dengeleme yapmadığı için en az karşılaştırma ile kurulur ve 0 swap yapar; ancak veriler kötü dağılmışsa yüksek (dengesiz) ağaç oluşur.
- **AVL:** her insertion sonrası dengeyi kontrol ettiği için karşılaştırma sayısı BST'ye yakındır, fakat **rotation (swap)** maliyeti vardır; karşılığında en sıkı yükseklik garantisi ($\approx 1.44 \cdot \log n$) elde edilir.
- **2-3 Ağacı:** bir düğümde 1 veya 2 anahtar tutabildiği için **toplam yükseklik en azdır**. Karşılaştırma sayısı AVL ile yarışır; "swap" yerine **split** sayılır.

Veri rastgele üretildiğinden her çalıştırmada sayılar değişebilir, ancak göreceli sıralama (BST swap=0, AVL ve 2-3 dengeleme maliyeti olur) genellikle korunur.